DLL DIGITAL SIGNING – IMPLEMENTATION GUIDE

1. Introduction

This document explains the process followed to digitally sign a .NET Class Library (DLL) using an Authenticode code-signing certificate.

The objective is to:

- Create a .NET Class Library project.
- Generate a code-signing certificate using Windows PowerShell.
- Sign the compiled DLL using Microsoft SignTool.
- Configure certificate trust on the local Windows machine.
- Verify that the digital signature was successfully applied.
- Confirm the signature through both command-line verification and Windows file properties.

This implementation serves as a Proof of Concept (POC) for understanding and implementing digital signing in .NET applications.

2. Tools and Technologies Used

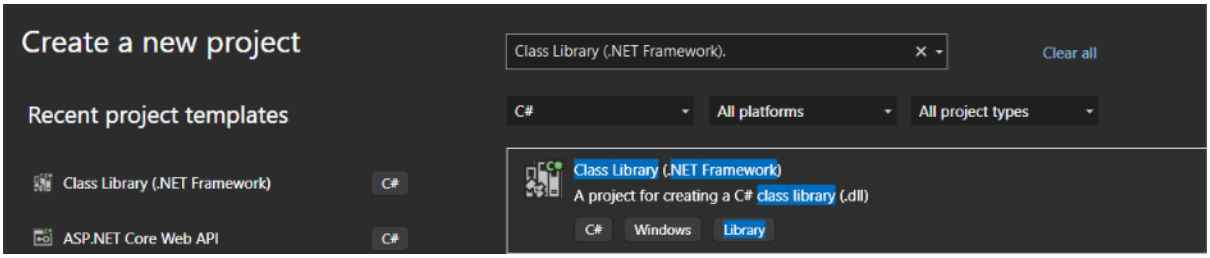
The following tools and technologies were used:

Tool / Technology	Purpose
Visual Studio	Create and build the .NET Class Library
C#	Develop the library functionality
.NET Framework 4.8	Target framework for the Class Library
Windows PowerShell	Generate and manage the code-signing certificate
Windows Certificate Store	Store the certificate and configure trust
Microsoft SignTool	Digitally sign and verify the DLL
Windows SDK	Provides the SignTool utility

3. Project Creation

Step 1: Create the Class Library Project

1. Open Visual Studio.
2. Select Create a new project.
3. Choose Class Library (.NET Framework).



4. Select C# as the programming language.
5. Set the project name to:

SignedDllPoc

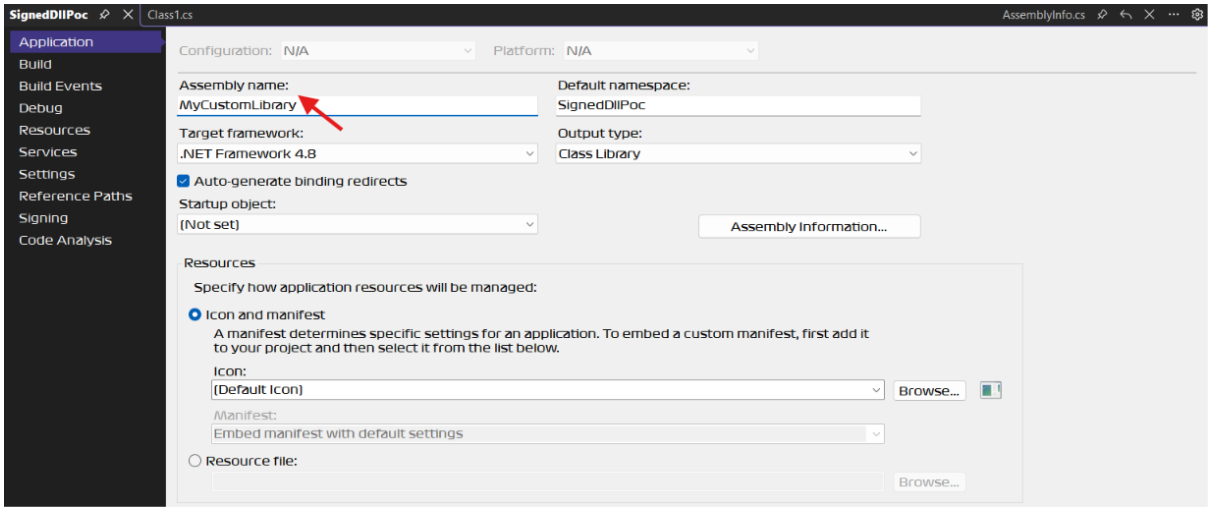
6. Select .NET Framework 4.8 as the target framework.
7. Create the project.

Step 2: Configure the Assembly Name

The assembly name determines the name of the compiled DLL.

1. Right-click the project in Solution Explorer.
2. Select Properties.
3. Open the Application tab.
4. Change the Assembly name to:

MyCustomLibrary



5. Save the changes.

The compiled DLL will now be named:

MyCustomLibrary.dll

Step 3: Add a Class to the Project

Rename the default Class1.cs file to:

GreetingService.cs

Add the following code:

```
namespace SignedDllPoc
{
  public class GreetingService
  {
    public string GetGreeting(string name)
    {
      if (string.IsNullOrWhiteSpace(name))
      {
        name = "Developer";
      }

      return "Hello " + name +
```

```
            "! This message comes from my custom signed DLL.";
    }
  }
}
```

This class provides a simple method that returns a greeting message.

The purpose of this class is to create a working DLL that can be used to demonstrate the digital-signing process.

Step 4: Build the Project

1. Select Build from the Visual Studio menu.
2. Click Build Solution.
3. Confirm that the build completes successfully.

The generated DLL is located at:

SignedDllPoc\bin\Debug\MyCustomLibrary.dll

Example path name:

C:\Users\<PC_NAME>\OneDrive\Documents\C#\SignedDllPoc\SignedDllPoc\bin\Debug\MyCustomLibrary.dll

At this stage, the DLL is compiled but has not yet been digitally signed.

4. Generate a Code-Signing Certificate

Step 5: Create a Self-Signed Certificate

A code-signing certificate is required to digitally sign the DLL.

For this POC, we generated a self-signed certificate using Windows PowerShell.

Open PowerShell and execute the following command:

```
$cert = New-SelfSignedCertificate `
  -Subject "CN=Cyberpunk" `
  -FriendlyName "Cyberpunk Code Signing Certificate" `
  -Type CodeSigningCert `
  -CertStoreLocation "Cert:\CurrentUser\My" `
  -KeyAlgorithm RSA `
  -KeyLength 3072 `
  -HashAlgorithm SHA256 `
  -NotAfter (Get-Date).AddYears(3)
```

Explanation of the Command

- **New-SelfSignedCertificate:** Creates a new self-signed certificate.
- **Subject:** Defines the certificate's identity. We used CN=Cyberpunk.
- **FriendlyName:** Sets a descriptive name for the certificate.
- **Type CodeSigningCert:** Specifies that the certificate is intended for code signing.
- **CertStoreLocation:** Stores the certificate in the current Windows user's Personal certificate store.
- **KeyAlgorithm RSA:** Uses the RSA public-key cryptographic algorithm.
- **KeyLength 3072:** Creates a 3072-bit RSA key.
- **HashAlgorithm SHA256:** Uses SHA-256 for the signing operation.
- **NotAfter:** Sets the certificate's expiration date to three years from creation.

Step 6: Verify the Certificate

After creating the certificate, verify that it exists in the Personal certificate store.

Execute:

```
Get-ChildItem Cert:\CurrentUser\My |
  Where-Object {
    $_.Subject -eq "CN=Cyberpunk"
  } |
  Format-List Subject, FriendlyName, Thumbprint, NotAfter, HasPrivateKey
```

Confirm the following:

- Subject is CN=Cyberpunk.
- FriendlyName is Cyberpunk Code Signing Certificate.
- HasPrivateKey is True.
- A certificate thumbprint is displayed.
- An expiration date is displayed.

The thumbprint identifies the certificate and is used later when signing the DLL.

The certificate created during our POC had the following thumbprint:

```
1C7C5426D0ED9BF54319787DD38E20E82909B5A5
```

Important: The certificate and its private key are stored in the current user's Windows certificate store. The private key is required to create digital signatures.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\devdi> $cert = New-SelfSignedCertificate `
>>      -Subject "CN=Cyberpunk" `
>>      -FriendlyName "Cyberpunk Code Signing Certificate" `
>>      -Type CodeSigningCert `
>>      -CertStoreLocation "Cert:\CurrentUser\My" `
>>      -KeyAlgorithm RSA `
>>      -KeyLength 3072 `
>>      -HashAlgorithm SHA256 `
>>      -NotAfter (Get-Date).AddYears(3)
PS C:\Users\devdi> Get-ChildItem Cert:\CurrentUser\My |
>>      Where-Object {
>>          $_.Subject -eq "CN=Cyberpunk"
>>      } |
>>      Format-List Subject, FriendlyName, Thumbprint, NotAfter, HasPrivateKey


Subject       : CN=Cyberpunk
FriendlyName  : Cyberpunk Code Signing Certificate
Thumbprint    : E80C643077C75AF78A4FFF11F46C230E23CCD8B0
NotAfter      : 11-11-2029 18:18:31
HasPrivateKey : True

Subject       : CN=Cyberpunk
FriendlyName  : Cyberpunk Code Signing Certificate
Thumbprint    : 1C7C5426D0ED9BF54319787DD38E20E82909B5A5
NotAfter      : 11-11-2029 13:07:29
HasPrivateKey : True
```

5.Locate Microsoft SignTool

Step 7: Install and Locate SignTool

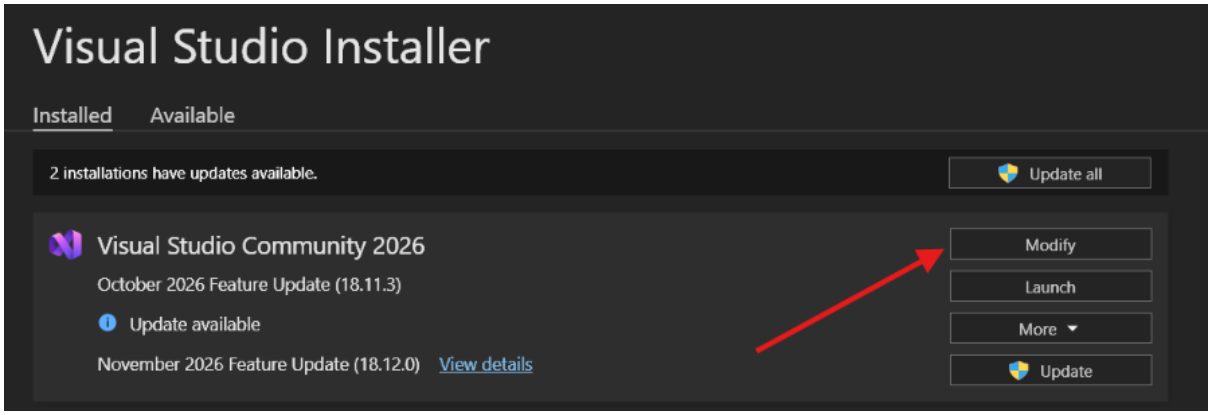
Microsoft SignTool is a command-line utility used to digitally sign files and verify their signatures.

It is included with the Windows SDK.

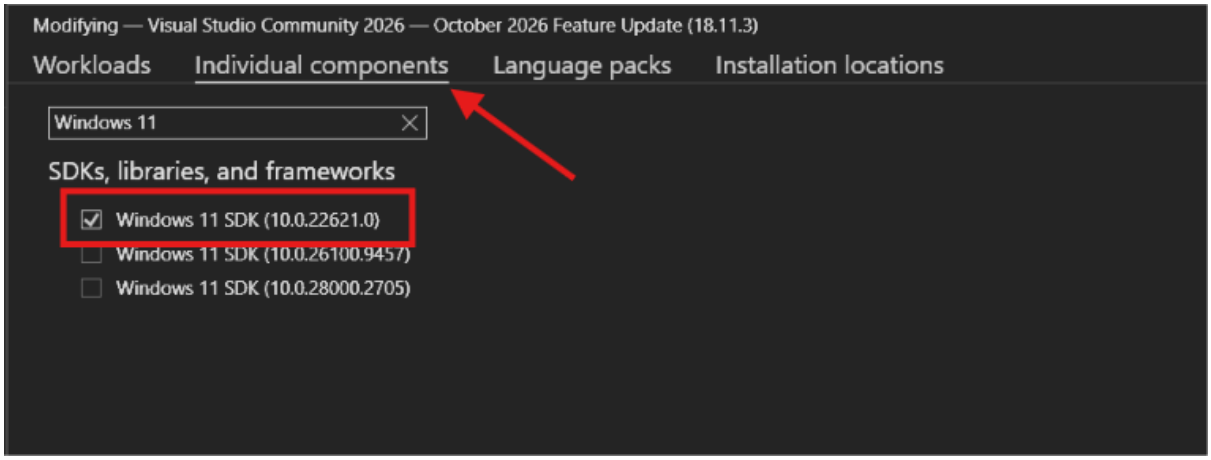
If SignTool is not available, install the Windows SDK through Visual Studio Installer by modifying the Visual Studio installation and selecting the relevant Windows SDK component.

Here are the following steps to install Windows SDK:

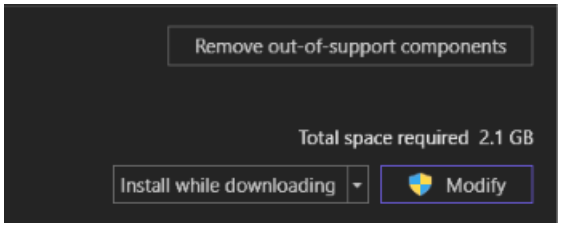
1.Open visual studio installer, click on Modify



2.Tap on Individual Components Tab,Search for Windows 10/11 SDK and check the box



3.Click on modify to install



In our environment, SignTool was located at:

C:\Program Files (x86)\Windows Kits\10\bin\10.0.22621.0\x64\signtool.exe

For convenience, define its location in PowerShell:

```
$signTool = "C:\Program Files (x86)\Windows Kits\10\bin\10.0.22621.0\x64\signtool.exe"
```

Verify that the executable exists:

```
Test-Path $signTool
```

Expected result:

True

```
PS C:\WINDOWS\System32> Test-Path $signTool
True
```

This confirms that PowerShell can locate SignTool at the specified path.

6. Digitally Sign the DLL

Step 8: Identify the Certificate and DLL

First, retrieve the code-signing certificate from the Personal certificate store.

```
$cert = Get-ChildItem Cert:\CurrentUser\My |
  Where-Object {
    $_.Subject -eq "CN=Cyberpunk" -and $_.HasPrivateKey
  } |
  Sort-Object NotAfter -Descending |
  Select-Object -First 1
```

This command selects the most recently expiring matching certificate that has a private key.

Next, define the DLL path:

```
$dll =
"C:\Users\devdi\OneDrive\Documents\C#\SignedDllPoc\SignedDllPoc\bin\Debug\MyCustomLibrary.dll"
```

Step 9: Execute the Signing Command

Execute the following command:

```
& $signTool sign /v /fd SHA256 /s My /sha1 $cert.Thumbprint $dll
```

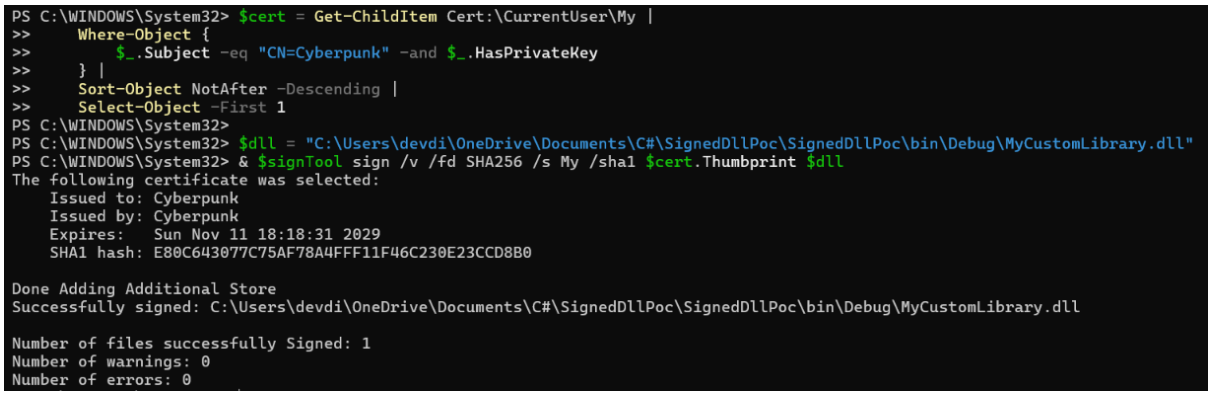
Explanation of the Signing Command

- **sign:** Tells SignTool to digitally sign the specified file.
- **/v:** Enables verbose output to display detailed signing information.
- **/fd SHA256:** Specifies SHA-256 as the file digest algorithm.
- **/s My:** Selects the Personal certificate store.

- **/sha1:** Identifies the certificate using its SHA-1 thumbprint.
- **$cert.Thumbprint:** Supplies the thumbprint of the selected certificate.
- **$dll:** Specifies the DLL that must be signed.

The certificate's SHA-1 thumbprint is used here as a certificate identifier. This does not mean that SHA-1 is being used as the file-signing digest. The file digest algorithm is SHA-256.

Expected Result



SignTool should display information indicating that the signing operation completed successfully.

At this point, the DLL contains a digital signature created using the Cyberpunk code-signing certificate.

However, successful signing and successful trust verification are two separate things. Because this certificate is self-signed, Windows does not automatically trust it.

7. Configure Certificate Trust

Step 10: Understand Why Trust Configuration Is Required

A self-signed certificate is not automatically trusted by Windows.

When we initially verified the DLL, SignTool identified the signing certificate but reported a certificate-chain trust error.

This happened because the certificate was created locally and was not issued by a publicly trusted Certificate Authority (CA).

To complete local verification, we added the public certificate to the appropriate Windows certificate stores.

Step 11: Export the Public Certificate

First, identify the certificate using its thumbprint:

```
$thumbprint = "1C7C5426D0ED9BF54319787DD38E20E82909B5A5"

$cert = Get-ChildItem Cert:\CurrentUser\My |
  Where-Object {
    $_.Thumbprint -eq $thumbprint -and $_.HasPrivateKey
  } |
  Select-Object -First 1
```

Define the export location:

```
$cerPath = Join-Path $env:TEMP "Cyberpunk-CodeSigning.cer"
```

Export the public certificate:

```
Export-Certificate -Cert $cert -FilePath $cerPath -Type CERT
```

This creates the following file:

Cyberpunk-CodeSigning.cer

```
PS C:\WINDOWS\System32> $thumbprint = "1C7C5426D0ED9BF54319787DD38E20E82909B5A5"
PS C:\WINDOWS\System32> $cert = Get-ChildItem Cert:\CurrentUser\My |
>>     Where-Object {
>>         $_.Thumbprint -eq $thumbprint -and $_.HasPrivateKey
>>     } |
>>     Select-Object -First 1
PS C:\WINDOWS\System32>
PS C:\WINDOWS\System32> $cerPath = Join-Path $env:TEMP "Cyberpunk-CodeSigning.cer"
PS C:\WINDOWS\System32> Export-Certificate -Cert $cert -FilePath $cerPath -Type CERT


    Directory: C:\Users\devdi\AppData\Local\Temp


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        11-11-2026     18:31           1020 Cyberpunk-CodeSigning.cer
```

The exported CER file contains the public certificate, not the private key.

Step 12: Import the Certificate into Trusted Stores

Import the certificate into the Current User Root Certification Authorities store:

```
Import-Certificate -FilePath $cerPath -CertStoreLocation "Cert:\CurrentUser\Root"
```

Next, import it into the Current User Trusted Publishers store:

```
Import-Certificate -FilePath $cerPath -CertStoreLocation
"Cert:\CurrentUser\TrustedPublisher"
```

Purpose of These Stores

Trusted Root Certification Authorities

This store establishes the certificate as a trusted root for the current Windows user. Since our certificate is self-signed, we explicitly trust it as a root certificate for this local POC.

Trusted Publishers

This store identifies publishers whose signed software is trusted by the current Windows user.

After importing the certificate into both stores, we can perform local signature verification without the earlier certificate-chain trust error.

Security Note: Trusting a self-signed certificate is an explicit security decision. In this POC, the certificate is trusted only on the current user's Windows account. It does not become a publicly trusted certificate.

8.Verify the Digital Signature

Step 13: Verify Using SignTool

Once the certificate trust configuration is complete, execute:

```
& $signTool verify /pa /v $dll
```

Explanation of the Verification Command

- **verify:** Requests verification of the file's digital signature.
- **/pa:** Uses the Windows Authenticode verification policy.
- **/v:** Displays detailed verification information.
- **$dll:** Specifies the DLL to verify.

Expected Verification Output

```
PS C:\WINDOWS\System32> $signTool = "C:\Program Files (x86)\Windows Kits\10\bin\10.0.22621.0\x64\signtool.exe"
PS C:\WINDOWS\System32>
PS C:\WINDOWS\System32> $dll = "C:\Users\devdi\OneDrive\Documents\C#\SignedDllPoc\SignedDllPoc\bin\Debug\MyCustomLibrary.dll"
PS C:\WINDOWS\System32>
PS C:\WINDOWS\System32> & $signTool verify /pa /v $dll

Verifying: C:\Users\devdi\OneDrive\Documents\C#\SignedDllPoc\SignedDllPoc\bin\Debug\MyCustomLibrary.dll

Signature Index: 0 (Primary Signature)
Hash of file (sha256): 29859412F6E207CF40AF5921FBBB1F0984BD72BFED0E5D21F32551D68E891E38

Signing Certificate Chain:
    Issued to: Cyberpunk
    Issued by: Cyberpunk
    Expires:   Sun Nov 11 13:07:29 2029
    SHA1 hash: 1C7C5426D0ED9BF54319787DD38E20E82909B5A5

File is not timestamped.


Successfully verified: C:\Users\devdi\OneDrive\Documents\C#\SignedDllPoc\SignedDllPoc\bin\Debug\MyCustomLibrary.dll
```

The output should contain information similar to the following:

Signature Index: 0 (Primary Signature)

Hash of file (sha256): ...

Signing Certificate Chain:

Issued to: Cyberpunk

Issued by: Cyberpunk

Expires: ...

SHA1 hash: ...

File is not timestamped.

Successfully verified:

MyCustomLibrary.dll

Number of files successfully Verified: 1

Number of warnings: 0

Number of errors: 0

How to Interpret the Output

The following indicators confirm successful verification:

1. **Signature Index: 0 (Primary Signature)**

Indicates that a primary digital signature is present.

2. **Hash of file (sha256)**

Displays the SHA-256 digest associated with the signed file.

3. **Issued to: Cyberpunk**

Confirms that the signing certificate belongs to the publisher identity configured during certificate creation.

4. **Issued by: Cyberpunk**

Indicates that the certificate is self-signed.

5. **Successfully verified**

Confirms that SignTool successfully verified the file's signature under the selected verification policy.

6. **Number of errors: 0**

Confirms that no verification errors were reported.

The exact hash and expiration date may differ between implementations because they depend on the actual file and certificate.

8. Verify Through Windows File Properties

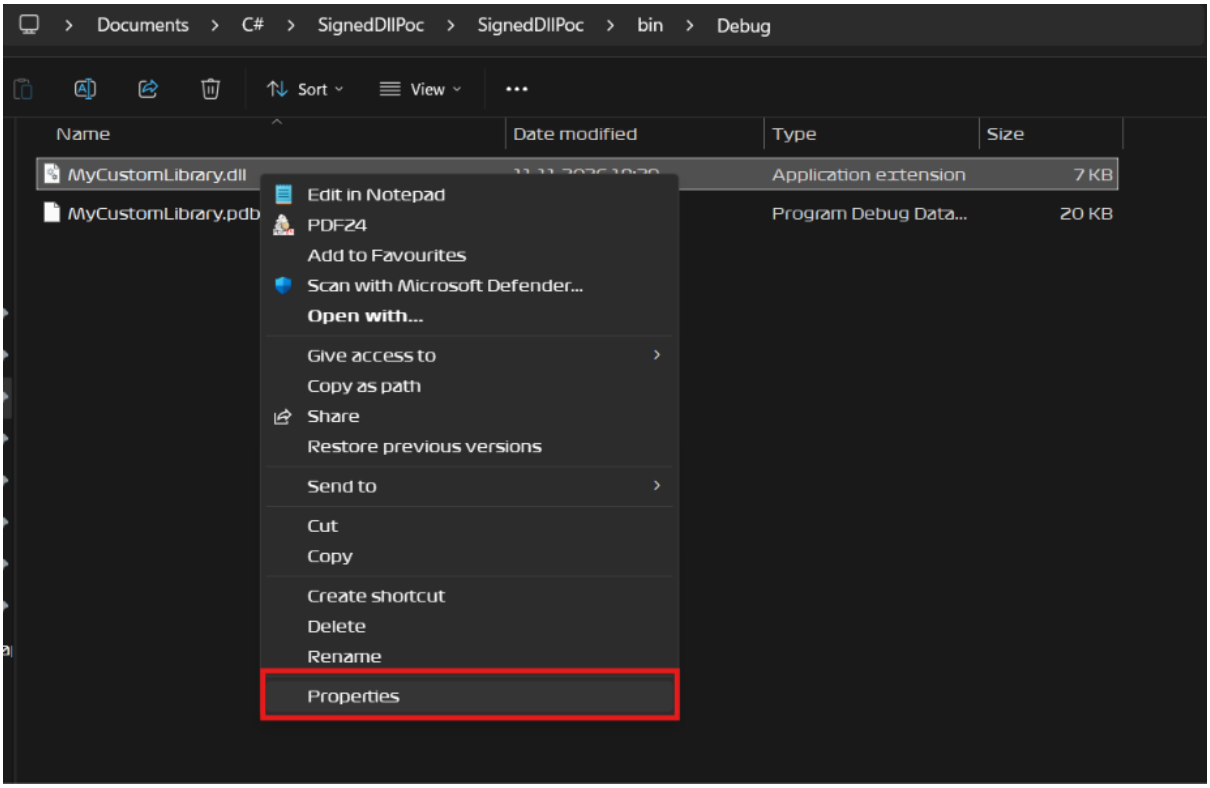
Step 14: Verify Using the Windows GUI

In addition to command-line verification, the signature can be checked through Windows File Explorer.

1. Navigate to the folder containing MyCustomLibrary.dll.

2. Right-click the DLL.

3. Select Properties.



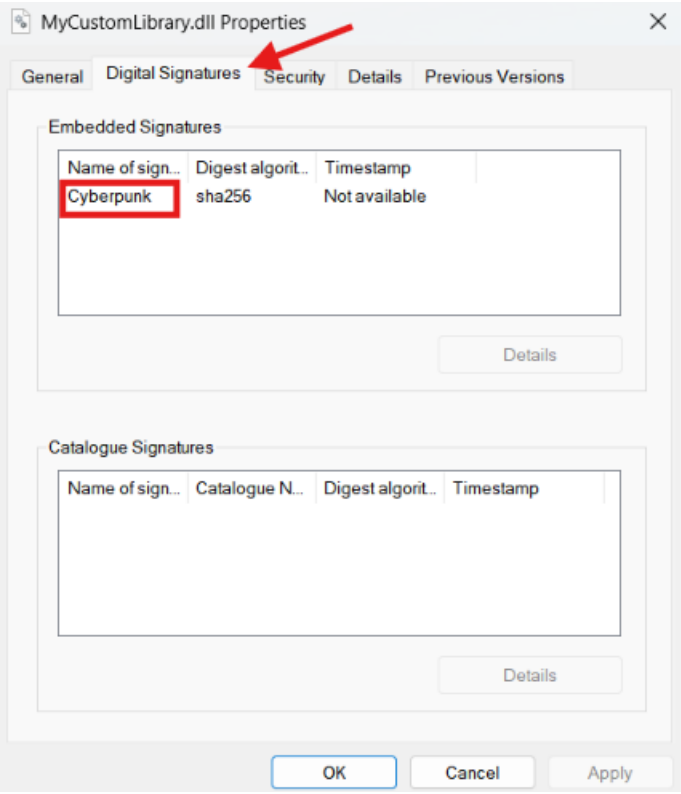
4. Open the Digital Signatures tab.

5. Select the displayed signature.

6. Click Details.

The Digital Signatures tab should display the signer as:

Cyberpunk



The signature details should indicate that the digital signature is valid.

This provides a convenient visual method for confirming the signature without executing PowerShell commands.

9. Understanding What the Digital Signature Does

A digital signature provides a way to verify the integrity and publisher identity associated with a file.

When a DLL is signed:

- A cryptographic digest of the file is calculated.
- The signing operation uses the certificate's private key.
- The signature is embedded in the DLL.
- Verification checks the signature against the file's contents and the corresponding certificate.
- Windows can evaluate whether the certificate is trusted under its configured trust policy.

If the signed file is modified after signing, signature verification will generally fail because the file's contents no longer match the signed digest.

A digital signature does not encrypt the DLL or prevent someone from opening, copying, or reverse-engineering it.

It also does not guarantee that the code is safe or free from vulnerabilities.

10. Important Considerations for Production Use

10.1 Self-Signed Certificate vs. CA-Issued Certificate

The certificate used in this POC is self-signed.

It is suitable for development, testing, and controlled internal demonstrations.

For software distributed to external customers or machines outside the organization, a certificate issued by a recognized Code Signing Certificate Authority is generally recommended.

A publicly trusted certificate helps recipients establish publisher trust without manually installing a self-signed root certificate.

10.2 Certificate Distribution

The public CER certificate can be distributed to authorized machines when an organization deliberately chooses to establish trust in an internal self-signed certificate.

The private key must never be distributed with the DLL, EXE, installer, or source code.

The private key must remain protected and accessible only to authorized signing personnel or secure signing infrastructure.

10.3 Rebuilding the DLL

Signing should be performed after the final DLL has been built.

If the project is rebuilt after signing, the newly generated DLL may replace the previously signed DLL.

Therefore, the correct sequence is:

1. Complete the code changes.
2. Build the project.
3. Confirm the final DLL is generated.
4. Sign the DLL.
5. Verify the signature.
6. Distribute the signed DLL.

10.4 Timestamping

During this POC, the DLL was signed without a timestamp.

The verification output displayed:

File is not timestamped.

A timestamp records when the signature was applied. For production signing, an appropriate trusted timestamp service is recommended.

Timestamping helps preserve signature validity beyond the signing certificate's expiration date, provided the signature and timestamp meet the relevant trust and validation requirements.

10.5 Signing Other Application Files

The same general approach can be used to sign other supported Windows executable formats, including:

- EXE files
- DLL files
- MSI installer packages

Each file must be signed separately.

Signing a DLL does not automatically sign an application EXE or its installer.

11. End-to-End Process Summary

The complete process followed in this POC is:

1. Create a C# Class Library project in Visual Studio.
2. Configure the assembly name as MyCustomLibrary.
3. Add the GreetingService class.
4. Build the project and generate MyCustomLibrary.dll.
5. Generate a self-signed code-signing certificate using PowerShell.
6. Store the certificate in the Current User Personal certificate store.
7. Locate Microsoft SignTool from the Windows SDK.
8. Sign the DLL using SHA-256 and the certificate's private key.
9. Export the certificate's public portion as a CER file.

10. Import the certificate into the Current User Root and Trusted Publishers stores.
11. Verify the DLL using SignTool.
12. Confirm the publisher identity and signature through Windows File Properties.

12. Final Outcome

The POC successfully demonstrated the digital signing of a .NET Framework 4.8 Class Library.

The following outcomes were achieved:

- A custom C# DLL was created and compiled successfully.
- A self-signed code-signing certificate was generated.
- The DLL was signed using Microsoft SignTool.
- The certificate was configured as trusted for the current Windows user.
- Command-line verification completed successfully.
- Windows File Properties displayed the Cyberpunk digital signature.

The final verification reported:

Successfully verified: MyCustomLibrary.dll

Number of files successfully Verified: 1

Number of warnings: 0

Number of errors: 0

This confirms that the digital-signing workflow was successfully implemented and verified in the local development environment.

13. Conclusion

This implementation establishes the foundation for integrating digital signing into future .NET development and deployment workflows.

The same principles can be applied when distributing internally developed libraries, application executables, and installer packages.

For production adoption, the organization should establish a controlled certificate-management process, protect private keys, use an appropriate trusted code-signing certificate, and incorporate signing and verification into the software release pipeline.